CTF Writeup

题目信息

附件里给了两个文件：

- `task.py`
- `cs_data.txt`

最终目标是恢复 `flag`。

题目整体分成三层：

1. 用椭圆曲线点序列加密 `random_key1`
2. 用弱 RSA 加密 `random_key2` 的 AES 轮密钥
3. 用 `key = random_key1 ^ random_key2` 作为 AES-CBC 密钥加密 flag

所以解题路线非常明确：

1. 先从椭圆曲线部分恢复 `random_key1`
2. 再从 RSA 部分恢复 `random_key2`
3. 异或得到 AES key，最后解密 `c_aes`

一、源码分析

核心代码如下：

```
G1, G2, G3, P1, P2, P3 = generate_base_points()

while True:
    a0 = random.randrange(0, o)
    b0 = random.randrange(0, o)
    c0 = random.randrange(1, o)
    K_point = a0 * P1 + b0 * P2 + c0 * G3
    if K_point != E(0):
        break
```

以及按 bit 加密的逻辑：

```
for bit in bits:
    a = random.randrange(0, o)
    b = random.randrange(0, o)
    c = random.randrange(0, o)
    if bit == '0':
        C = a * P1 + b * P2 + c * G3
    else:
        C = a * P1 + b * G2 + c * P3
    cs.append(C.xy())
```

RSA 和 AES 相关部分：

```
p = generate_prime(1024)
q = getPrime(1024)
n = p * q
e = 65537

random_key1 = os.urandom(16)
random_key2 = os.urandom(16)

key = bytes([a ^ b for a, b in zip(random_key1, random_key2)])

cs = encrypt_bytes_with_weil(random_key1, K_point, P1, P2, P3, G1, G2, G3, o)

last_key = key_expansion(random_key2)
c_rsa = pow(bytes_to_long(last_key), e, n)

iv = os.urandom(16)
cipher = AES.new(key, AES.MODE_CBC, iv)
c_aes = cipher.encrypt(pad(flag, 16))
```

二、椭圆曲线部分

题目给出了：

```
o =
7934793907292155125165079512831690660881306799603939520592833378730174535830236823
67384822284289
n1 = 859267
n2 = 52437899
n3 = 28355811
```

其中：

```
n3 = 28355811 = 3 * 11 * 859267 = 33 * n1
```

对 `o` 做分解可以得到：

```
o = 3 * 11 * 10177 * 859267 * 52437899 * r
```

其中：

```
r = 524358751751261904794477405081859658376905525005276378226036586999385811845 13
```

于是令：

```
M = r * n2 * n3
```

那么：

- `P1` 的阶是 `n1`，而 `n1 | n3`，所以 `M * P1 = O`
- `P2` 的阶是 `n2`，所以 `M * P2 = O`

- `P3` 的阶是 `n3`，所以 `M * P3 = O`
- `G2`、`G3` 的阶都是 `o`，所以 `M * G2`、`M * G3` 的阶都变成

```
o / M = 10177
```

也就是说，乘上 `M` 以后，所有信息都会被压缩到一个很小的 `10177` 阶子群里。

题目里先放了一个头部点：

```
K_point = a0 * P1 + b0 * P2 + c0 * G3
```

乘上 `M`：

```
M * K_point = c0 * (M * G3)
```

因为前两项都被杀掉了，所以 `M * K_point` 一定落在由 `M * G3` 生成的那个 `10177` 阶子群里。记这个子群为 `H`。

再看每个 bit 对应的点：

如果 bit 是 `0`：

```
C = a * P1 + b * P2 + c * G3
M * C = c * (M * G3) ∈ H
```

如果 bit 是 `1`：

```
C = a * P1 + b * G2 + c * P3
M * C = b * (M * G2)
```

这里 `M * G2` 也是一个 `10177` 阶点，但它一般不会落在 `H` 里。由于 `10177` 是素数，小子群碰撞到同一个循环子群里的概率大约只有 `1 / 10177`，可以认为是极小概率事件。

所以判定规则就是：

- `M * C ∈ H`，判成 bit `0`
- `M * C ∉ H`，判成 bit `1`

把 `cs[0]` 先乘 `M` 得到一个生成元：

```
S = M * K_point
```

然后枚举 `S` 生成的整个子群：

```
H = {0, S, 2S, 3S, ..., 10176S}
```

对每个密文点 `C` 判断 `M * C` 是否在 `H` 中，就能直接得到 128 个 bit。

最终恢复出：

```
random_key1 = 8df563a823b3add6e0a4da31f2555ebe
```

三、RSA 部分

1. 漏洞点

题目中的 `p` 不是普通随机素数，而是这样生成的：

```
def generate_prime(bits):
    while True:
        p_sub = 2
        for prime in sieve_base:
            p_sub *= prime
            if p_sub.bit_length() > bits - 2:
                break

        for k in range(2, 10000, 2):
            p = p_sub * k + 1
            if isPrime(p):
                return p
```

这意味着：

```
p - 1 = p_sub * k
```

其中 `p_sub` 是一大堆小素数的乘积，`k < 10000`，所以 `p - 1` 非常光滑，天然适合 `Pollard p-1`。

---

2. Pollard p-1 分解

直接做 `Pollard p-1`，很快就能拿到一个因子 `p`，进而求出 `q = n / p`。

之后就能按标准 RSA 流程解密：

```
phi = (p - 1) * (q - 1)
d = e^{-1} mod phi
last_key = c_rsa^d mod n
```

这里得到的不是 `random_key2`，而是：

```
last_key = key_expansion(random_key2)
```

也就是 AES-128 密钥扩展后的最后 4 个 word。

实际恢复为：

```
last_key = 780bedbd0bea8eec7cd4326719550006
```

---

3. 逆 AES 密钥扩展

因为 AES-128 的 key schedule 是可逆的，所以我们可以从最后一轮轮密钥反推初始 16 字节密钥。

最终恢复出：

```
random_key2 = 51ded1088be2bfd82291a04a42f6cd32
```

四、AES 解密

题目里的对称密钥是：

```
key = random_key1 ^ random_key2
```

带入上面两部分的结果：

```
random_key1 = 8df563a823b3add6e0a4da31f2555ebe
random_key2 = 51ded1088be2bfd82291a04a42f6cd32
key         = dc2bb2a0a851120ec2357a7bb0a3938c
```

然后用题目给的 `iv` 和 `c_aes` 做 AES-CBC 解密即可，得到：

```
dart{0d7c228a-0c75-4914-a272-ca2c96cd5ca7}
```

五、完整 exp

```
import ast
from math import gcd
from Crypto.Cipher import AES
from Crypto.Util.Padding import unpad


p_ec =
0x1a0111ea397fe69a4b1ba7b6434bacd764774b84f38512bf6730d2a0f6b0f6241eabfffeb153fff
fb9feffffffffaaab
o =
79347939072921551251650795128316906608813067996039395205928333787301745358302368 2
367384822284289
n1 = 859267
n2 = 52437899
n3 = 28355811
```

```
n =
14685041170442204918405583160343810361127399864157434453915718782247011711119244
1
82284028542609253159049444356051277435936122276004110281047525142001296734525331
0
50807531366987425538514646370537662417484609069332729969644444729935781621823595
3
97751335601084907375438838006662717394642316000585253171946870348239514974012675
0
23883975621298790267393334061627819356473932940398241429724450013327830686972706
8
94699526743313331517030094447522772668816841405008374828190777036088750626239079
2
67510008813105616250329668805860316320441298839560869256942099758202223732009421
9
469054284986095923802700263866210516176584677461256983
c_rsa =
74231514119138617803996275719518567011575633515104945151215552023045037756701780
5
26260789061976755658325418843994780806559238907177047055473949449639134295693377
5
84046710895240662122821981567353777690859611002522560646785856006740342985854904
6
95610475038620729237499009395199224056982292128775251998161390549676312977578250
9
89501803335573269265794244598105444939510585700441331525728789380327207222547771
5
75094896363209815599271832390676431405381466011192298785646752634683579526124097
3
56770748495318358864466168591744819079230251579490595399596992924889730056630225
2
828997693662370952748590478015242943666108852664993 30
iv = bytes.fromhex("9ee38737a1bdaaa5f665b4d91988e8d6")
c_aes =
bytes.fromhex("8533aeeb80395a4dc344ca9e4fe036463e6563b3acc3ad85087c166a4e497c516b
8cda153dbe3ab2d59999d61195b16e")
e = 65537

SBOX = [
    0x63, 0x7c, 0x77, 0x7b, 0xf2, 0x6b, 0x6f, 0xc5, 0x30, 0x01, 0x67, 0x2b, 0xfe,
0xd7, 0xab, 0x76,
    0xca, 0x82, 0xc9, 0x7d, 0xfa, 0x59, 0x47, 0xf0, 0xad, 0xd4, 0xa2, 0xaf, 0x9c,
0xa4, 0x72, 0xc0,
    0xb7, 0xfd, 0x93, 0x26, 0x36, 0x3f, 0xf7, 0xcc, 0x34, 0xa5, 0xe5, 0xf1, 0x71,
0xd8, 0x31, 0x15,
    0x04, 0xc7, 0x23, 0xc3, 0x18, 0x96, 0x05, 0x9a, 0x07, 0x12, 0x80, 0xe2, 0xeb,
0x27, 0xb2, 0x75,
    0x09, 0x83, 0x2c, 0x1a, 0x1b, 0x6e, 0x5a, 0xa0, 0x52, 0x3b, 0xd6, 0xb3, 0x29,
0xe3, 0x2f, 0x84,
    0x53, 0xd1, 0x00, 0xed, 0x20, 0xfc, 0xb1, 0x5b, 0x6a, 0xcb, 0xbe, 0x39, 0x4a,
0x4c, 0x58, 0xcf,
    0xd0, 0xef, 0xaa, 0xfb, 0x43, 0x4d, 0x33, 0x85, 0x45, 0xf9, 0x02, 0x7f, 0x50,
0x3c, 0x9f, 0xa8,
    0x51, 0xa3, 0x40, 0x8f, 0x92, 0x9d, 0x38, 0xf5, 0xbc, 0xb6, 0xda, 0x21, 0x10,
0xff, 0xf3, 0xd2,
    0xcd, 0x0c, 0x13, 0xec, 0x5f, 0x97, 0x44, 0x17, 0xc4, 0xa7, 0x7e, 0x3d, 0x64,
0x5d, 0x19, 0x73,
    0x60, 0x81, 0x4f, 0xdc, 0x22, 0x2a, 0x90, 0x88, 0x46, 0xee, 0xb8, 0x14, 0xde,
0x5e, 0x0b, 0xdb,
    0xe0, 0x32, 0x3a, 0x0a, 0x49, 0x06, 0x24, 0x5c, 0xc2, 0xd3, 0xac, 0x62, 0x91,
0x95, 0xe4, 0x79,
    0xe7, 0xc8, 0x37, 0x6d, 0x8d, 0xd5, 0x4e, 0xa9, 0x6c, 0x56, 0xf4, 0xea, 0x65,
0x7a, 0xae, 0x08,
    0xba, 0x78, 0x25, 0x2e, 0x1c, 0xa6, 0xb4, 0xc6, 0xe8, 0xdd, 0x74, 0x1f, 0x4b,
0xbd, 0x8b, 0x8a,
    0x70, 0x3e, 0xb5, 0x66, 0x48, 0x03, 0xf6, 0x0e, 0x61, 0x35, 0x57, 0xb9, 0x86,
0xc1, 0x1d, 0x9e,
    0xe1, 0xf8, 0x98, 0x11, 0x69, 0xd9, 0x8e, 0x94, 0x9b, 0x1e, 0x87, 0xe9, 0xce,
0x55, 0x28, 0xdf,
```

```
    0x8c, 0xa1, 0x89, 0x0d, 0xbf, 0xe6, 0x42, 0x68, 0x41, 0x99, 0x2d, 0x0f, 0xb0,
0x54, 0xbb, 0x16
]
RCON = [0x00, 0x01, 0x02, 0x04, 0x08, 0x10, 0x20, 0x40, 0x80, 0x1b, 0x36]


def inv_mod(a, m):
    return pow(a, -1, m)


def ec_add(P, Q):
    if P is None:
        return Q
    if Q is None:
        return P
    x1, y1 = P
    x2, y2 = Q
    if x1 == x2:
        if (y1 + y2) % p_ec == 0:
            return None
        lam = (3 * x1 * x1) * inv_mod((2 * y1) % p_ec, p_ec) % p_ec
    else:
        lam = (y2 - y1) * inv_mod((x2 - x1) % p_ec, p_ec) % p_ec
    x3 = (lam * lam - x1 - x2) % p_ec
    y3 = (lam * (x1 - x3) - y1) % p_ec
    return (x3, y3)


def ec_mul(k, P):
    R = None
    Q = P
    while k:
        if k & 1:
            R = ec_add(R, Q)
        Q = ec_add(Q, Q)
        k >>= 1
    return R


def sub_word(w):
    return (
        (SBOX[(w >> 24) & 0xff] << 24)
        | (SBOX[(w >> 16) & 0xff] << 16)
        | (SBOX[(w >> 8) & 0xff] << 8)
        | SBOX[w & 0xff]
    )


def rot_word(w):
    return ((w << 8) & 0xffffffff) | (w >> 24)


def inv_key_expansion(last_key):
    w = [0] * 44
    for i in range(40, 44):
        w[i] = int.from_bytes(last_key[4 * (i - 40):4 * (i - 39)], "big")
```

```python
    for i in range(43, 3, -1):
        temp = w[i - 1]
        if i % 4 == 0:
            temp = sub_word(rot_word(temp)) ^ (RCON[i // 4] << 24)
        w[i - 4] = w[i] ^ temp
    return b"".join(x.to_bytes(4, "big") for x in w[:4])


def pollard_pm1(n, B=10000):
    a = 2
    for j in range(2, B + 1):
        a = pow(a, j, n)
        g = gcd(a - 1, n)
        if 1 < g < n:
            return g
    raise ValueError("factor not found")


with open("cs_data.txt", "r", encoding="utf-8") as f:
    cs = ast.literal_eval(f.read())


# step 1: recover random_key1
r = 52435875175126190479447740508185965837690552500527637822603658699938581184513
M = r * n2 * n3

S = ec_mul(M, cs[0])
assert S is not None

subgroup = set()
T = None
for _ in range(10177):
    subgroup.add(T)
    T = ec_add(T, S)

bits = []
for C in cs[1:]:
    T = ec_mul(M, C)
    bits.append("0" if T in subgroup else "1")

random_key1 = int("".join(bits), 2).to_bytes(16, "big")
print("[+] random_key1 =", random_key1.hex())


# step 2: recover random_key2
p = pollard_pm1(n)
q = n // p
phi = (p - 1) * (q - 1)
d = inv_mod(e, phi)
last_key = pow(c_rsa, d, n).to_bytes(16, "big")
random_key2 = inv_key_expansion(last_key)
print("[+] last_key    =", last_key.hex())
print("[+] random_key2 =", random_key2.hex())


# step 3: AES decrypt
```

```python
key = bytes(a ^ b for a, b in zip(random_key1, random_key2))
print("[+] aes key     =", key.hex())

cipher = AES.new(key, AES.MODE_CBC, iv)
flag = unpad(cipher.decrypt(c_aes), 16)
print("[+] flag        =", flag.decode())
```

六、运行结果

```
[+] random_key1 = 8df563a823b3add6e0a4da31f2555ebe
[+] last_key    = 780bedbd0bea8eec7cd4326719550006
[+] random_key2 = 51ded1088be2bfd82291a04a42f6cd32
[+] aes key     = dc2bb2a0a851120ec2357a7bb0a3938c
[+] flag        = dart{0d7c228a-0c75-4914-a272-ca2c96cd5ca7}
```

七、最终答案

```
dart{0d7c228a-0c75-4914-a272-ca2c96cd5ca7}
```